



Dùng AI Claude Code để code game Unity

Live Demo: Xây game "Star Defender" từ đầu đến cuối bằng AI

AI đang thay đổi cách làm game

52%

game dev dùng AI
(GDC 2025, 3,000+ dev)

73%

studio đã áp dụng AI
(a16z Games Survey)

37%

indie studio dẫn đầu
vs 21% AAA/AA

- Indie dùng AI gấp đôi AAA — lợi thế cho team nhỏ
- Top use cases: playtesting (47%), localization (45%), code generation (44%)
- McKinsey: dev hoàn thành tasks nhanh gấp 2x với AI, documentation giảm 50%
- Câu hỏi: một dev + AI có thể build game từ đầu đến cuối?

Claude Code là gì?



AI agent chạy trong terminal — đọc toàn bộ codebase (1M tokens)



Chạy compile, test, git — bất kỳ lệnh nào



Tự iterate: sinh code → chạy → lỗi → tự sửa → chạy lại



46% developer chọn là tool #1 — vượt Cursor (19%), Copilot (9%)

\$1 tỷ

doanh thu/năm trong 6 tháng • **4%** tổng GitHub commits

So sánh với các AI coding tool khác

Tiêu chí	ChatGPT	Copilot	Cursor	Claude Code
Scope project	File paste	8K–16K tokens	File đang mở	1M tokens
Multi-file edit	Không	Hạn chế	Có	Mạnh nhất
Tự chạy lệnh	Không	Không	Hạn chế	Có
Tự sửa lỗi	Không	Không	Một phần	Vòng lặp tự động

Giờ hãy xem Claude Code hoạt động thực tế — chúng ta sẽ build game "Star Defender" từ đầu đến cuối

Star Defender — Game chúng ta sẽ build



2D Space Shooter • Top-down • Wave-based

1 PlayerShip

Di chuyển, bắn đạn, iFrames

2 Enemies

Zigzag, Bomber, AI patterns

3 Combat

Damage, HP, collision

4 Waves

15 waves, boss mỗi 5 wave

5 Power-ups

Shield, rapid fire, bomb

6 Polish

UI, editor tools, balance

Mỗi feature = 1 prompt cho Claude Code. Hãy bắt đầu.

Bước 0: Setup — File CLAUDE.md

CLAUDE.md (Đặt ở root project)

Star Defender — Project Context cho AI

Tech Stack

- Unity 6.0, C# 12, .NET 8
- UniTask (KHÔNG Coroutine), Zenject (KHÔNG Singleton)
- New Input System (KHÔNG Input cũ), DOTween

Architecture

- 2D Space Shooter, camera top-down, scroll dọc
- Object pooling: BulletPool, EnemyPool (Zenject)
- Event-driven: IEventBus (SignalBus)
- Wave-based: WaveConfig (ScriptableObject)

Conventions

- Namespace: StarDefender.{Feature}
- [SerializeField] private, prefix _ cho private fields
- Interface: IDamageable, IPoolable, IShootable

KHÔNG LÀM

- GameObject.Find(), FindObjectOfType()
- transform.Translate() → dùng Rigidbody2D.MovePosition()
- Resources.Load() → dùng Addressables

File này = bộ não cho AI. Claude Code đọc đầu tiên, tuân thủ xuyên suốt mọi prompt sau

Feature 1: PlayerShip — Viết prompt

• • • Terminal

> **Tạo PlayerShip controller cho Star Defender:**

- Di chuyển A/D hoặc Arrow (chỉ trục X), giới hạn trong viewport
- Bắn đạn bằng Space, fire rate 0.2s, đạn bay lên
- Dùng Rigidbody2D (Kinematic), object pooling cho đạn
- Có iFrames 1.5s khi bị trúng đạn, nhấp nháy sprite
- Tách ShipWeapon.cs riêng cho shooting logic
- [SerializeField] tắt cả, namespace StarDefender.Player



Tại sao prompt này tốt?

- Component cụ thể: Rigidbody2D Kinematic, không transform.Translate
- Giá trị cụ thể: fire rate 0.2s, iFrames 1.5s
- Kiến trúc: tách ShipWeapon.cs — Đúng Single Responsibility

Feature 1: Code sinh ra

PlayerShip.cs (output)

```
namespace StarDefender.Player {
    public class PlayerShip : MonoBehaviour,
        IDamageable {
        [Inject] IBulletPool _bulletPool;
        [Inject] GameBounds _bounds;

        [SerializeField] float _moveSpeed = 10f;
        [SerializeField] float _iFrames = 1.5f;
        Rigidbody2D _rb;
        bool _isInvincible;

        void Update() {
            float h = _input.Move.ReadValue<float>();
            var pos = _rb.position +
                Vector2.right * h * _moveSpeed
                * Time.deltaTime;
            _rb.MovePosition(
                _bounds.ClampPosition(pos));
        }
    }
}
```

Claude Code tự động:

- Phát hiện Zenject → dùng [Inject] thay Singleton
- Tìm thấy GameBounds có sẵn → dùng ClampPosition()
- Phát hiện New Input System → dùng InputAction
- Tạo thêm ShipWeapon.cs (tách riêng)
- Tạo IDamageable interface dùng chung

AI đọc CLAUDE.md + project hiện tại → sinh code tuân thủ convention, dùng lại code có sẵn

Feature 1: Lỗi compile → AI tự sửa

• • • Terminal

> (Claude Code phát hiện lỗi và tự sửa)

✗ CS0117: 'InputActionReference' does not contain 'ReadValue'
at PlayerShip.cs line 18

🔍 InputActionReference cần .action.ReadValue<T>(), không gọi trực tiếp

🔧 Sửa: `_input.Move.action.ReadValue<float>()`
+ thêm `_input.Fire.action.performed += OnFire` trong `OnEnable()`
+ thêm `OnDisable()` unsubscribe để tránh memory leak

✓ Compile OK. 2 files sửa. 0 errors, 0 warnings.

Kết quả Feature 1: 3 files tạo + 1 file sửa. Tổng ~25 phút thay vì 4-6 giờ viết tay

Feature 2: Enemies — 2 loại trong 1 prompt

• • • Terminal

> Tạo 2 enemy types cho Star Defender:

- BasicEnemy: bay thẳng xuống, bắn mỗi 2s
- ZigzagEnemy: bay sin wave, bắn mỗi 1.5s
- Cả 2 kế thừa EnemyBase (abstract class)
- 3 HP, drop PowerUp 20% khi chết
- Dùng EnemyPool (object pooling)

EnemyBase.cs (output)

```
public abstract class EnemyBase :  
    MonoBehaviour, IDamageable, IPoolable {  
    [Inject] IEventBus _events;  
    protected abstract void Move();  
    public void TakeHit(int dmg) {  
        _hp -= dmg;  
        if (_hp <= 0) {  
            _events.Fire(new EnemyKilled(this));  
            if (Random.value < 0.2f)  
                SpawnPowerUp();  
        } } }  
}
```

- AI tự tạo abstract class + 2 concrete classes + integrate với pool system
- Tự implement IDamageable (đã tạo ở Feature 1) — code consistency
- Output: EnemyBase.cs, BasicEnemy.cs, ZigzagEnemy.cs — ~15 phút

Feature 3: Combat — Chia nhỏ task

✗ "Làm hệ thống combat hoàn chỉnh" — quá rộng, AI phải đoán

✓ Chia thành 4 prompts nhỏ — mỗi cái ~10 phút:

- Prompt 1: "Tạo Bullet.cs: bay lên 15f, tự deactivate khi ra viewport, IPoolable"
- Prompt 2: "Tạo DamageSystem: IDamageable.TakeHit() trả int, event OnDamaged"
- Prompt 3: "Tạo CollisionHandler: OnTriggerEnter2D, layer Player/Enemy/Bullet"
- Prompt 4: "Tạo HitVFX: particle burst khi trúng đạn, object pool, 0.5s lifetime"

McKinsey: task đơn giản ~50% nhanh hơn, task phức tạp chỉ <10%. Chia nhỏ = luôn ở nhóm 50%

Bug: Wave 2 crash — Claude Code debug

• • • Terminal

> Wave 2 bắt đầu nhưng game crash: `NullReferenceException`

at `WaveSpawner.SpawnEnemy()` line 67

at `WaveSpawner.<RunWave>d__12.MoveNext()` line 45

 Đọc `WaveSpawner.cs`, `EnemyFactory.cs`, `WaveConfig.json`...

 Nguyên nhân: `WaveConfig` wave 2 có `enemyType = "Bomber"` nhưng `EnemyFactory` chỉ handle "Basic" và "Zigzag". Bomber chưa register → null → `NullRef` khi spawn.

 Fix:

1. Thêm `BomberEnemy.cs` kế thừa `EnemyBase` (thả bomb thay bắn)
2. Register trong `EnemyFactory` + null check + `Debug.LogError`

✓ 3 files sửa, 1 file tạo. Wave 2 chạy OK.

AI đọc 3 files cùng lúc, trace root cause trong 2 phút — dev đọc từng file mất 15–30 phút

Feature 4: 15 Waves — Generate data config

• • • Terminal

> Tạo 15 wave configs:

- Difficulty curve logarithmic
- Boss mỗi 5 wave (wave 5, 10, 15)
- Mix enemy types tăng dần

WaveConfigs.json (output)

```
[{ "wave":1, "enemies":[
  {"type":"Basic","count":4},
  "spawnRate":2.0 },
{ "wave":2, "enemies":[
  {"type":"Basic","count":5},
  {"type":"Zigzag","count":2}],
  "spawnRate":1.8 },
// ... wave 3-4 tự tăng
{ "wave":5, "enemies":[
  {"type":"Zigzag","count":6},
  {"type":"Bomber","count":3}],
  "boss":"MotherShip" },
// ... đến wave 15 final boss
]
```

- 15 waves hoàn chỉnh trong 5 phút — hand-design mất nửa ngày
- Import thẳng vào ScriptableObject, test ngay trong Unity
- Dễ tinh chỉnh: sửa JSON → reload → test lại

Refactor: Tách God Class giữa chừng

TRƯỚC: 1 file làm mọi thứ

```
// StarDefenderManager.cs — 500 dòng
public class StarDefenderManager {
    public void SpawnWave() { ... }
    public void AddScore() { ... }
    public void UpdateUI() { ... }
    public void PauseGame() { ... }
    public void SaveHighScore() { ... }
    // God class — khó maintain
}
```

SAU: 5 files tách biệt

// SAU: 5 files rõ trách nhiệm

ScoreManager.cs	→ Điểm + combo
WaveSpawner.cs	→ Spawn logic
GameUIController.cs	→ HUD updates
GameStateManager.cs	→ Pause/Resume
HighScoreService.cs	→ Save/Load

```
// Tất cả inject qua Zenject
// Test riêng từng module được
```

- Claude Code sửa 8 files đồng thời, redirect references, giữ logic
- Từ Singleton → Zenject DI — testable, maintainable
- Thời gian: ~20 phút refactor thay vì 2–3 giờ sợ break code

Feature 5: Power-ups — Prompt tệ vs tốt

• • • Terminal

```
> Làm power-up cho game  
→ Power-up gì? Hiệu ứng gì?  
→ Pickup ra sao? Kéo dài bao lâu?  
→ Phải iterate 5+ lần
```

• • • Terminal

```
> Tạo ShieldPowerUp cho PlayerShip:  
- Drop 20% khi enemy chết, OnTriggerEnter2D  
- Shield 3 hit, kéo dài 10s, VFX orbit  
→ Đúng ngay lần đầu!
```

- 2 phút viết prompt kỹ tiết kiệm 20 phút sửa code sai
- Công thức: Component cụ thể + giá trị cụ thể + data flow + convention
- Áp dụng cho RapidFirePowerUp và BombPowerUp tương tự

Tiếp: dùng /read để thêm shield vào PlayerShip.cs đang có — không phá code cũ



Feature 5 (tiếp): /read → thêm Shield

• • • Terminal

> /read Assets/Scripts/Player/PlayerShip.cs

(Đọc 95 dòng hiện tại...)

> Thêm shield: khi nhặt ShieldPowerUp, ship có 3 hit shield, 10s. Absorb damage thay HP.



Phân tích PlayerShip.cs hiện tại:

- Có TakeHit() dùng UniTask → sửa thêm shield check
- Có _isInvincible flag → shield interact với iFrames
- Convention: _shieldHits, _shieldTimer (giữ _prefix)



Thêm vào PlayerShip.cs:

- ActivateShield(int hits, float duration) → public
- Sửa TakeHit(): if (_shieldHits > 0) absorb thay giảm HP
- Spawn ShieldVFX orbit, destroy khi hết shield

Claude Code đọc code CŨ → thêm tính năng MỚI tương thích — không phá logic đang chạy tốt

Feature 6: Polish — Editor Tool preview wave

Editor/WaveConfigEditor.cs (output)

```
[CustomEditor(typeof(WaveConfig))]  
public class WaveConfigEditor : Editor {  
    public override void OnInspectorGUI() {  
        DrawDefaultInspector();  
        if (GUILayout.Button("> Preview Wave")) {  
            WavePreviewWindow.Show((WaveConfig)target);  
        }  
        if (GUILayout.Button("⚖️ Auto-Balance")) {  
            var c = (WaveConfig)target;  
            c.enemyCount = Mathf.RoundToInt(3 + c.waveNumber * 1.8f);  
            c.spawnRate = Mathf.Max(0.5f, 2.5f - c.waveNumber * 0.12f);  
            EditorUtility.SetDirty(target);  
        }  
    }  
}
```

- Preview spawn points trong Scene View + auto-balance difficulty
- Claude Code rất giỏi Editor scripting — pattern rõ ràng, API Ổn Định
- Loại tool dev cần nhưng không muốn tốn thời gian viết tay

Star Defender — Hoàn thành!



12+

C# scripts
tạo bởi AI

~3 giờ

tổng thời gian
(thay vì 2–3 ngày)

6

features
hoàn chỉnh

1

bug fix
AI tự trace + sửa

Step	Thời gian
Setup CLAUDE.md	10 phút
PlayerShip + weapon	25 phút
2 enemy types	15 phút
Combat system (4 prompts)	40 phút
Bug fix wave 2	5 phút
15 waves config	5 phút
Refactor God class	20 phút
3 power-ups	30 phút

Case Study: Void Balls — 10 ngày, \$50/tháng

10

ngày

29K

dòng C#

173

scripts

\$50

AI/tháng

- LoopedIn Games: 2D roguelite bullet-hell hoàn chỉnh
- 5 enemy types, 15 power-ups, boss fights — production-ready
- Claude Code chạy 8 agent song song + Unity 6 + Unity MCP
- Quy mô tương đương Star Defender x10 — và chỉ 10 ngày

Enterprise: Claude Code ở production

Rakuten

24→5 ngày

Deliver feature

Wiz

20 giờ

50K dòng Python→Go

Ramp

-80%

Thời gian debug

Altana

2-10x

Dev velocity

Giới hạn — Bức tranh trung thực

1.7x

nhiều lỗi hơn

45%

chứa lỗi hổng

3x

readability tệ hơn

19.7%

package ảo

- Không mở Unity Editor — vẫn cần tay set reference, prefab, scene
- Không hiểu gameplay feel — "đạn thiếu punch" cần con người
- METR: dev giàu kinh nghiệm thực tế chậm 19% với AI (do review time)
- 45% dev nói debug code AI tốn hơn tự viết — kỹ năng prompt quyết định

Dùng AI đúng cách



Chia nhỏ task + prompt cụ thể = tiết kiệm 50% thời gian



Dùng CLAUDE.md để AI hiểu project — giảm 80% iterate



Dùng /read để thêm feature vào code cũ — không phá logic



Không dùng cho kiến trúc tổng thể — bạn phải là architect

Star Defender: ~3 giờ AI + bạn = game hoàn chỉnh. Bạn design, AI implement.

Chi phí: Bao nhiêu để build Star Defender?

Tool	Free	Standard	Power User
Claude Code	Không	\$20/tháng	\$100–200/tháng
Copilot	2K completions	\$10/tháng	\$39/tháng
Cursor	2K completions	\$20/tháng	\$60–200/tháng

- Star Defender build với Claude Pro \$20/tháng — đủ cho indie dev
- Power user: Claude Max \$100–200/tháng, trung bình ~\$6/dev/ngày
- So sánh: thuê junior dev VN ~\$800–1,200/tháng
- ROI: 2.5–3.5x trung bình, top 4–6x (Larridin 2026)

3 bài học từ Star Defender

CLAUDE.md

= bộ não cho AI. Setup 10 phút, tiết kiệm hàng giờ iterate.
AI tuân thủ convention xuyên suốt mọi prompt.

Chia nhỏ

= luôn thắng. Mỗi prompt 1 task rõ ràng, giá trị cụ thể.
~50% nhanh hơn cho task đơn giản (McKinsey).

/read

= vũ khí bí mật. Thêm feature mới vào code cũ
mà không phá logic đang chạy tốt.

Call to action: Tạo CLAUDE.md cho project của bạn → thử 1 task nhỏ → iterate

Thị trường AI Gaming

\$5.85B

AI trong game 2024

\$37.89B

Dự kiến 2034
+20.54%/năm

\$1.8B

VC đầu tư AI gaming
178 startups

- Unity chiếm 51% game trên Steam — ecosystem lớn nhất cho AI
- 29% developer tin AI giúp team nhỏ cạnh tranh ngang studio lớn
- AI startups: từ 4% lên ~65% gaming infrastructure investment
- Không phải trend — là chuyển đổi cơ bản của ngành



Q&A

Chi phí?

\$20–200/tháng, trung bình \$6/ngày

Code AI có bug?

1.7x nhiều hơn — luôn review

Multiplayer?

Logic OK, networking cần KN dev

Bắt đầu từ đâu?

Tạo CLAUDE.md → 1 task nhỏ → iterate